LOTUSim: Multi-Domain Simulator for Marine Robotics

Cédric Buche[1,3], Juliette Grosset[1,2], Hélène Lechêne[2], Marie Dubromel[1,2],
Pierig Havez-Bodivit[2], Malcom Neo[2], Julien Prodhon[2]

[1]CROSSING IRL 2010, CNRS; [2]Naval Group, France; [3]IMT Atlantique

***Abstract*— Simulation is essential for maritime robotics, supporting operator training, mission rehearsal, and human–vehicle interaction in environments where real-world testing is costly or hazardous. Existing simulators focus primarily on autonomy systems and often lack human-in-the-loop interaction and realistic environmental physics. This paper introduces LOTUSim, an open-source, real-time maritime simulator supporting multi-user interaction across aerial, surface, and underwater robotic systems for coordinated naval-style operations. The first contribution of this work is enabling real-time interactive performance for users while ensuring scalability to large fleets operating within a shared interactive simulation environment. Validation demonstrates robust human-in-the-loop performance, maintaining strict real-time execution and high visual fidelity while scaling to large heterogeneous maritime drone swarms. The second contribution is a computationally efficient, Ekman-inspired layered, underwater current model that captures wind-driven, depth-dependent flow dynamics with sufficient physical fidelity for large-scale simulations. Validation against ocean reanalysis data demonstrates substantially improved accuracy compared to commonly used stochastic Gauss–Markov current models. These results confirm LOTUSim's suitability as a simulation platform for operator-in-the-loop maritime robotics research.**

I. Introduction

Simulation is a foundational capability for modern maritime operations, supporting system design, operator training, mission rehearsal, and risk reduction in environments where real-world testing is costly, hazardous, or operationally constrained. For naval operations, human-in-the-loop (HITL) simulation is essential. Maritime missions are conducted under strong environmental forcing, limited sensing capabilities, and high cognitive workload, where operator decisions influence mission success and safety. Interactive simulation interfaces can significantly improve situational awareness, skill acquisition, and crew coordination by allowing operators to interact naturally with vehicles and their environment [1].

Despite recent advances, existing marine robotic simulators are designed primarily for autonomous systems and sensor simulation, with limited consideration for real-time operator-centric interaction and collaborative mission execution. Even simulators actively maintained and updated in 2025, such as OceanSim [2], MarineGym [3], Stonefish [4] and HoloOcean [5], [6], do not yet provide native support for immersive, interactive operation or multi-user mission rehearsal involving human operators in the loop.

Environmental modelling represents a second major barrier to operational realism in existing marine robotic simulators. Many simulators neglect underwater currents or use simplified models: USVsim [7] relies on Computational Fluid Dynamics (CFD)-based precomputed fields, LRAUVSim [8] and MarineGym [3] uses a constant unidirectional current, and StoneFish [4] allows arbitrary profiles without physical realism. HoloOcean [6] plans to implement more realistic volumetric currents, though currently only user-defined vector fields (vortex fields) are supported. The Gauss–Markov process, implemented in UUVSim [9] and stratified in DAVE [10], provides a real-time, computationally efficient baseline. While practical, these approaches are insufficient for naval operations, where depth-dependent, wind-driven currents strongly affect vehicle behaviour, sensor performance, and operator decision-making.

This paper presents **LOTUSim**[1,2], an open-source multi-domain maritime simulator (Figure 1), designed to support naval-style operations and involving both human operators and robotic systems. LOTUSim targets training, mission rehearsal in complex maritime environments, where environmental forcing, limited observability, and operator workload play a central role. To this end, the simulator enables real-time interaction between operators and heterogeneous robotic platforms across aerial, surface, and underwater domains within a distributed execution framework.

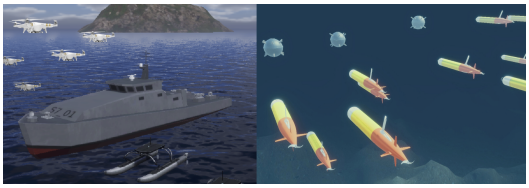

Fig. 1: LOTUSim, an open multi-domain simulator

The main contributions of *LOTUSim* are:

- **A framework enabling real-time interaction**, demonstrating robust interactive performance and scalability for large heterogeneous fleets while maintaining real-time responsiveness.
- **A high-fidelity underwater current model** for real-time simulation, with an Ekman-inspired layered current model, validated against ocean reanalysis data and outperforming Gauss–Markov baseline models.
- **A comprehensive suite of robotic and immersive features**, integrating diverse onboard sensors (LiDAR, Radar, camera) with human-centric interfaces (Head Mounted Display, eye-tracking, Leap Motion).

[1]LOTUSim open-source code: https://github.com/naval-group/LOTUSim
[2]LOTUSim Video: https://www.youtube.com/watch?v=iXDz8ZqSpq4

II. Review of Marine Robotic Simulators

Physics simulators for maritime applications allow researchers to experiment in a controlled environment before deploying. Additionally, simulations can be executed faster than real-time, which is particularly beneficial for learning-based approaches [8]. In this context, this section provides a review of maritime simulators in robotics, focusing on their ability to simulate aerial, surface, and underwater conditions. Although aerial, surface, and underwater environments are connected, their distinct physical properties demand different simulation models. Aerial domains involve wind and turbulence, while surface and underwater environments require modelling of buoyancy, waves, and hydrodynamic forces. To address these differences clearly, the review is divided into two parts: aerial simulations first, followed by surface and underwater domains.

A. Aerial

Table II categorises various simulators according to the criteria defined in Table I. In the context of a real-time maritime simulation platform for human-vehicle interaction, the expected feature is primarily real-time models for dynamic adaptation with optional interest for wind variation (time and space), obstacle interaction, boundary restrictions. Many simulators, such as AirSim [11], employ simplified wind models that do not account for time and space dependent variability. Gazebo [12] and Stonefish [4] incorporates dynamic wind modelling based on time- and space-dependent variations[3]. Models based on CFD, such as OpenFOAM [13], excel in simulating wind interactions with structures (e.g., obstacles) [14], and are adopted by simulators such as USVsim [7]. However, CFD has notable limitations, including its reliance on pre-computed data [14]. In conclusion, Gazebo's or Stonefish's wind modelling offers a practical and efficient solution.

TABLE I: Criteria for Aerial modelisation

Type	Description
Wind Dependencies	Wind variation over time and space, enabling simulation of gusts and regional wind changes.
Obstacle Interaction	Wind behaviour around obstacles like buildings and terrain, capturing deflection and turbulence.
Bound Restrictions	Limits wind simulations to a predefined area, common in CFD models for controlled analysis.
Pre-compute	Precomputed wind fields (e.g., CFD) for static analysis or real-time models for dynamic adaptation.

TABLE II: Aerial Simulation Properties

Properties	AirSim	OpenFOAM	Gazebo	Stonefish	LOTUSim
Wind dependencies	✗	Space	Time + Space	Time + Space	Time + Space
Obstacle Interaction	✗	✓	✗	✗	✗
No Bound Restricted	✓	✗	Optional	Optional	Optional
Compute online	✓	✗	✓	✓	✓

[3] https://github.com/gazebosim/gz-sim/tree/gz-sim9/src/systems/wind_effects

B. Surface and Underwater

Table III provides a comparative overview of representative maritime simulators spanning surface and underwater domains. Recent simulators such as HoloOcean 2.0 [5], [6], Stonefish [4], MarineGym [3], OceanSim [15], and UNavSim [16] significantly advance the state of the art in terms of rendering quality, physics fidelity, and scalability. However, most existing platforms remain specialised either in surface or underwater operations, and mature cross-domain solutions remain limited. Several widely used simulators, including UUVsim [9], DAVE [10] and URSim [17], are no longer actively maintained, which restricts their long-term applicability. While more recent simulators address maintenance, performance, and visual realism, most lack native support for immersive HITL interaction and collaborative multi-user operation. In Table III, Immersive HITL refers to operator interaction through immersive interfaces such as VR, motion tracking, or embodied control, rather than conventional keyboard or graphical interfaces. Multi-user denotes the ability for multiple human operators to simultaneously interact within a simulation environment, which is distinct from multi-robot or multi-agent simulation[4].

These capabilities are essential for training, mission rehearsal, and human–vehicle interaction, yet remain largely absent from current maritime simulators, including recent state-of-the-art platforms. While ROS2 support and improved physics engines are increasingly adopted, the lack of immersive and collaborative interaction highlights a gap in the literature. LOTUSim is designed to address this gap by combining cross-domain simulation, realistic environmental modelling, immersive HITL interaction, and native multi-user support within a unified and actively maintained framework.

III. LOTUSim Architecture

A. Overview

LOTUSim leverages ROS2, Gazebo, and Unity as solutions for a distributed multi-agent simulation (Figure 2). ROS2 manages the multi-agent communication and helps integrate with real drones through customised plugins, while Gazebo provides realistic physics simulation. Unity is integrated as the primary visualisation and interaction engine. The primary goal of LOTUSim is to provide a distributed server-client-based simulation framework applicable across multiple domains, with a particular emphasis on marine use cases. Gazebo serves as the central orchestrator for asset management, while various interfaces and client modules are used to execute specific simulation models on demand. The core module (multi-agent simulation control) is interfaced with three other client modules: 1/ Physics computation 2/ Rendering 3/ Agent Interaction. Each interface supports different communication protocols. Currently, LOTUSim supports a wide range of protocols, including ROS2, WebSocket, and TCP/IP.

[4] Following Ferber [18], a Multi-Agent System (MAS) consists of autonomous agents operating in a distributed environment with synchronized communication. In this context, MAS support refers to dynamic agent addition and removal across distributed nodes, combined with a fair and adaptive scheduler that mitigates execution-order bias, dynamically allocates computational time, enables simulation time scaling, and supports real-time operator interaction.

TABLE III: Comparative overview of representative maritime simulators and supported features. (Legend: S = Surface, UW = Underwater, GM = Gauss–Markov.)

	UUVsim	URSim	USVsim	DAVE	LRAUVSim	UNavSim	OceanSim	MarineGym	StoneFish	HoloOcean	LOTUSim
Reference	[9]	[17]	[7]	[10]	[8]	[16]	[15]	[3]	[4]	[5], [6]	/
Domain	UW	UW	S	UW	UW	UW	UW	UW	S+UW	S+UW	S+UW
Launching	2016	2019	2019	2022	2023	2023	2025	2025	2019	2022	2025
Maintained	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
ROS2	✗	✗	✗	✗	✗	✓	✓	✗	✓	✓	✓
MAS	✓	✗	✗	✓	✓	✗	✗	✗	✓	✓	✓
Physics server	Gazebo	Unity	Gazebo	Gazebo	Gazebo	AirSim	Isaac Sim	Isaac Sim	Bullet	Unreal Engine	Gazebo
Rendering	Gazebo	Unity	Gazebo	Gazebo	Gazebo	Unreal Engine	Omniverse RTX	Omniverse RTX	OpenGL	Unreal Engine	Unity
Multi-user	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Immersive HITL	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Ocean current	GM	✗	CFD nonuniform currents	GM and stratified	Constant unidirectional force	✗	✗	Constant unidirectional force	Uniform, Jet and Pipe	Vortex current field	Ekman layered model

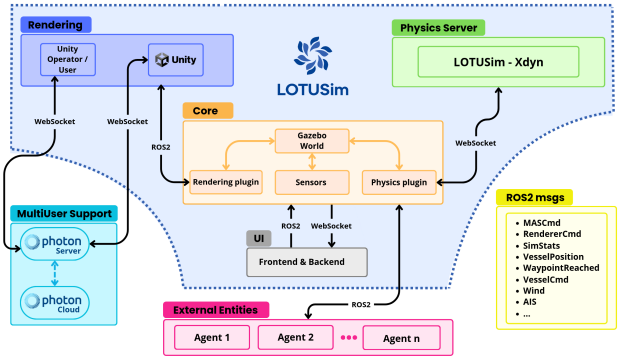

Fig. 2: System architecture of LOTUSim

When using Gazebo as the asset orchestrator, each of the main client types is associated with a dedicated Gazebo plugin responsible for querying the corresponding client to update the asset state. During each simulation update cycle in Gazebo, where the timestep length is user-configurable, each asset may issue a query to its respective client module, depending on simulation requirements. The client processes the request and returns the computed result. This architecture enables computational workloads to be offloaded to external machines. The update loop concludes once all client responses are received, ensuring synchronisation across the distributed system.

B. Technical Details

Physics computation: LOTUSim offloads physics calculations by transmitting each asset's position, velocity, and the current timestep duration to an external physics engine. Since different physics engines require varying data formats and parameters, we developed a base physics interface that provides a standardised method for extracting all relevant asset information. This abstraction simplifies integration with a wide range of external physics engines. To mitigate scheduling bias, caused by Gazebo's default behavior of updating assets in their load order, the physics interface randomises the update order of assets in each simulation loop. Additionally, due to LOTUSim's distributed architecture, where users may also be located on remote machines, ship control commands are transmitted via communication protocols (e.g., ROS2). This communication latency further randomises the timing of physics updates, promoting a more discrete and non-deterministic simulation environment.

Xdyn[5] is an open-source lightweight ship simulator that models real-time vessel dynamics at sea. For surface ships and underwater drones, we introduce an extension named as LOTUSim-Xdyn[6], detailed in Section IV, which connects to the simulation via WebSocket.

Agent interaction: Currently interfaced by the ROS2 system, agent interaction can be implemented in various programming languages by communicating using ROS2 DDS with a system through different action servers and topics.

Rendering and multi-user interaction: Visualisation is decoupled from the core simulation logic to support headless execution for AI training. When enabled, users can choose what to render and which renderer to integrate, but LOTUSim uses Unity as its default renderer. While engines like Unreal offer superior photorealism, Unity was selected for its superior extensibility and native support for immersive interfaces (e.g., VR and Leap Motion). In each timestep, the plugin will send out the locations of all assets, the newly added and destroyed assets are also published. Users are able to add custom rendering processes (e.g. explosion) into the visualisation system.

To facilitate shared missions, LOTUSim incorporates Photon Unity Networking (PUN2). This enables distributed state synchronisation, allowing multiple operators in different locations to interact with the same agents in real-time. By leveraging a cloud-relay architecture, PUN2 ensures consistent world-state replication across all Unity clients.

[5]Xdyn: https://gitlab.com/sirehna_naval_group/sirehna/xdyn
[6]LOTUSim-Xdyn: https://github.com/naval-group/LOTUSim-Xdyn

C. Multi-Agent Architecture

The multi-agent approach improves robustness and flexibility by allowing each entity to operate independently with local decision-making. Many works have extended Gazebo for multi-agent system (MAS) research by integrating with ROS-based frameworks [19]. Its flexible plugin architecture supports scalable MAS prototyping in applications such as cooperative exploration, formation control, and multi-robot task allocation.

The architecture is designed to support a flexible and distributed MAS. Each machine in the network can host one or more agents, and agents can be dynamically added or removed at runtime. The MAS plugin serves as a centralised controller within Gazebo. It hosts a ROS2 action server that supports both bulk and individual spawning of vessels. Users can interact with this server either through the LOTUSim UI or by developing custom scripts. During each simulation update loop, the system processes user-specified commands to spawn, delete, or move vessels accordingly. Similar to the physics update mechanism, execution order and timing are inherently randomised due to network communication delays, ensuring nondeterministic behavior and robustness in distributed environments.

D. Evaluation

The evaluation of LOTUSim focuses on its capacity for HITL interaction, specifically measuring how the system maintains responsiveness and visual fidelity as the complexity of the swarm increases.

Metrics for Interactivity: We evaluate the simulator through four key metrics that directly impact human perception and control precision:

- *Visual Frame Rate* - Frames Per Second (`FPS`) [20] assess visual continuity. For high-stakes HITL tasks, maintaining high FPS is vital to prevent operator spatial disorientation.
- *Physics Update Parameters* (`update_rate`) [8], [21] monitors the physics engine's heartbeat. A stable update rate ensures that vehicle physics react to human inputs without lag.
- *Real-Time Factor* (`RTF`) [22] is the ratio of simulation time to real-time. A consistent `RTF (=1)` is required for meaningful human intervention.

Results for Real-Time Interactive Performance: Using the configuration in Table IV, LOTUSim demonstrates a robust operational envelope for complex maritime scenarios, as summarised in Table V.

- *HITL Responsiveness*: To evaluate interactive control, we measured the $\overline{\texttt{update\_rate}}$ using 200 ms as the threshold for perceptually responsive operation. LOTUSim remained within this limit while scaling up to 750 Long-Range Autonomous Underwater Vehicles (LRAUVs) or 450 BlueROVs.
- *Physics Latency*: At a 30 ms control loop (the threshold update limit), LOTUSim can simulate swarms of up to 30 BlueROV or 55 LRAUV agents while keeping physics updates strictly synchronised with real-time.
- Visual Rendering: Across the same configurations, LOTUSim consistently maintained a high rendering rate ($\overline{\texttt{FPS}} > 140$), providing smooth and immersive visual feedback for operators.
- Large-Scale Heterogeneous Scenarios: Under strict real-time constraints (`RTF=1`), the simulator successfully manages a large-scale heterogeneous fleet, including heterogeneous surface, underwater and aerial vessels.

TABLE IV: Benchmark system specifications

Specification	LOTUSim
OS - Ubuntu	22.04.5 LTS
Processor	i9-13980HX
CPU	Up to 5.6 GHz
GPU NVIDIA GeForce RTX	4090 Laptop
GPU VRAM	16 GB
NVIDIA Driver	570.133.07
CUDA	12.8
Python	3.10.12
Gazebo	Harmonic
ROS	ROS2 Humble

TABLE V: Summary of Real-Time Interactive Performance

Evaluation Aspect	Observed Performance
HITL Responsiveness	750 LRAUVs or 450 BlueROVs
Physics Latency	30 BlueROVs or 55 LRAUVs
Visual Rendering	$\overline{\texttt{FPS}} > 140$
Large-Scale Heterogeneous Scenario	Surface: 1 DTMB, 3 commando boats, 3 WAMVs Underwater: 45 LRAUVs, 45 BlueROVs Aerial: 90 X500s

IV. Environment Modelling

Environmental effects such as wind, waves, and underwater currents are often modeled using real data or simplified formulations. However, real data lacks flexibility in the context of real-time or scenario-driven simulations. LOTUSim is a multi-domain maritime simulator that jointly models surface, underwater, and aerial dynamics, enabling realistic cross-domain interactions. It leverages Gazebo's wind modeling and introduces custom surface and underwater models, including a layered ocean current model inspired by Ekman layer theory [23], [24].

A. Surface

LOTUSim-Xdyn computes ship motion using Fossen's equations of motion [25] while incorporating detailed hydrodynamic forces, including Froude-Krylov forces and diffraction forces. It also features customisable maneuvering models and actuators. Some recent developments have been made to wind propulsion models for cargo ships [26].

For wave simulation, LOTUSim-Xdyn uses Airy wave theory, a linear mathematical model that describes gravity wave propagation on the ocean surface [27], [28]. This theory assumes uniform water depth and homogeneous fluid properties, providing an efficient foundation for wave-structure interaction calculations. LOTUSim-Xdyn allows users to

export environment data as 2D or 3D grids for post-analysis, including surface elevation.

B. Underwater

Several simulators adopt alternative strategies to represent ocean currents, each with specific limitations (see Table III). USVsim [7] relies on CFD-based precomputed fields, which prevent real-time simulation and limit dynamic adaptation. LRAUVSim [8] and MarineGym [3] implements a constant, unidirectional current, offering computational simplicity but neglecting realistic temporal or spatial variations. In contrast, StoneFish [4] supports completely arbitrary current profiles (uniform, jet, or pipe flows), providing flexibility for testing but lacking physical realism. Efforts such as HoloOcean [6] aim to introduce volumetric effects, but ocean currents are currently implemented using user-defined vector (e.g., vortex) fields. A widely used computational baseline is the Gauss–Markov (GM) process, which, despite lacking a physical foundation, is computationally efficient and suitable for real-time simulation. It serves as the standard real-time current model in UUVSim [9] and DAVE [10], making it a convenient baseline for comparisons.

The Ekman layer framework [29], [30] offers a practical method for simulating ocean currents. By combining wind measurements with classical Ekman theory, it delivers computationally efficient current estimates suitable for operational use [31], [32]. In our implementation, the water column is divided into three vertical zones under the assumption of steady-state flow. At the surface, wind stress generates currents that, under the influence of the Coriolis force, form the well-known Ekman spiral. Near the seabed, friction gives rise to a bottom Ekman layer, also exhibiting spiral patterns, while variations in bathymetry drive vertical motions. Between these layers lies the geostrophic interior, where the flow is largely in geostrophic balance and unaffected by surface or bottom friction.

Within LOTUSim-Xdyn, we incorporated two current representations: the standard constant-current model and the new Ekman-inspired formulation described here.

Parameters:

- Coriolis parameter: $f = 2\Omega \sin(\varphi)$, where φ is the latitude and Ω is Earth's rotation rate.
- Drag coefficient: We use a simple surface drag coefficient C_D inspired by Curcic and Haus [33]:

$$C_D = \begin{cases} (0.79 + 0.08\, U_{10}) \times 10^{-3}, & U_{10} < 20.5 \text{ m/s}, \\ 2.43 \times 10^{-3}, & U_{10} \geq 20.5 \text{ m/s}. \end{cases} \quad (1)$$

- Wind stress $\tau_s = C_D \rho_{\text{air}} U_{10}^2$ uses the 10 m wind U_{10}.

Note that u, v, w are functions of x, y, z and t.

Top layer:

Surface currents combine Airy wave orbital velocities with the Ekman spiral (northern hemisphere formulation):

$$\begin{aligned} u &= \bar{u} + V_0 e^{-\pi z_s / D_s} \cos\left(\frac{\pi}{4} - \frac{\pi z_s}{D_s} - \phi\right) \\ v &= \bar{v} + V_0 e^{-\pi z_s / D_s} \sin\left(\frac{\pi}{4} - \frac{\pi z_s}{D_s} + \phi\right) \\ w &= 0 \end{aligned} \quad (2)$$

where:

- z_s is the depth, null at the surface and positive downwards
- D_s is the surface Ekman layer depth
- V_0 is the current velocity at the surface
- ϕ is the wind orientation at the surface

Middle Layer:

$$u = \bar{u}, \quad v = \bar{v}, \quad w = 0 \quad (3)$$

The middle layer is unaffected by the waves or the seabed.

Bottom Layer:

For non-flat seabeds, correcting terms are introduced to preserve continuity and boundary conditions. In the northern hemisphere, the bottom Ekman spiral is given by

$$\begin{aligned} u &= \bar{u}\left[1 - e^{-\pi z_b / D_b} \cos\left(\frac{\pi z_b}{D_b}\right)\right] - \bar{v} e^{-\pi z_b / D_b} \sin\left(\frac{\pi z_b}{D_b}\right) \\ v &= \bar{u} e^{-\pi z_b / D_b} \sin\left(\frac{\pi z_b}{D_b}\right) + \bar{v}\left[1 - e^{-\pi z_b / D_b} \cos\left(\frac{\pi z_b}{D_b}\right)\right] \\ w &= 0 \end{aligned} \quad (4)$$

where:

- z_b is the depth, null at the seabed and positive upwards
- D_b is the bottom Ekman layer depth

C. Evaluation

This section assesses the effectiveness of the underwater current models implemented within LOTUSim.

Methodology and Setup: The evaluation involves a statistical comparison between Copernicus in situ measurements (real-world data), the Gauss–Markov current model (UUVsim, DAVE), and the Ekman layer–based model (LOTUSim).

- *Study Area*: The analysis focuses on waters off the coast of Brest, France (longitude: $[-6.25°, -6°]$, latitude: $[46.6°, 47°]$), chosen due to its dynamic underwater currents and the availability of high-quality observational datasets.
- *Depth Coverage*: Observational data span from 0.5 meters down to 1000 meters, capturing the range most relevant to surface and near-surface current dynamics.
- *Temporal Sampling*: Measurements were recorded at four times per day (00:00, 06:00, 12:00, and 18:00) over five separate days throughout the year. Each day represents different wind and environmental conditions, ensuring a diverse set of oceanographic scenarios. These selected days were then used to forecast conditions for the following day (see Table VI for details).

TABLE VI: Data variability

Variable	Minimum	Maximum	Mean
u (zonal current)	-0.52	0.55	0.004
v (meridional current)	-0.49	0.33	0.005
V_{north} (northward wind)	-12.6	14.9	-2.7
V_{east} (eastward wind)	-9.6	22.3	1.2

Dataset: The analysis utilises a total of 2,800 measurement points (N) distributed across the study area and depth

range, with 700 points corresponding to each observation time. For model validation, data from the following day were employed as a reference to assess predictions generated by both the Gauss–Markov model and the Ekman layer model (LOTUSim).

Evaluation Metrics: Model accuracy was quantified using both absolute and relative error measures for the zonal (u) and meridional (v) current components. Specifically, the mean absolute error (MAE) and root mean square error (RMSE) were employed, as these are widely adopted in ocean current forecasting studies [34], [35], [36]. Definitions for these metrics are provided in Table VII. These indicators enable a thorough comparison of model performance across different depths, observation times, and varying environmental conditions.

Results: Table VIII presents a comparative assessment of the Ekman layer model and the Gauss–Markov model. Across all depths and time intervals, the Ekman-based approach consistently lowers MAE and RMSE, with overall improvements ranging from approximately 40% to 85%. The most pronounced gains are observed in subsurface layers (0–100 m), where error reductions frequently exceed 70%. Mid-depth zones (100–500 m) exhibit the largest enhancements, with I_{MAE} and I_{RMSE} often surpassing 75%, highlighting the model's effectiveness in capturing subsurface current dynamics. Deeper layers also benefit, showing typical improvements above 60%, while surface layer results are more variable, with moderate gains around 40–55%.

Normalised error ratios further confirm that the Ekman model achieves lower relative MAE and RMSE compared to the Gauss–Markov approach. Overall, these findings indicate that incorporating Ekman dynamics enables more accurate representation of wind-driven and Coriolis-influenced currents, enhancing the simulation of large-scale underwater current dynamics.

V. Features of the Simulator

A. Models

LOTUSim provides open-source models, including:

- Underwater vehicles: LRAUV, BlueROV
- Surface vessels: DTMB surface ship, PHAs, FREMM, Commando boat, WAM-V
- Aerial drones: X500
- Environmental assets: mines, island, trees
- Human/robot avatars (Unity Animator handles motion and transitions)

B. Robotics Sensors

This subsection details the sensor suites integrated into the simulator, encompassing both onboard perception for autonomous systems and HITL sensing modalities. All sensor data are published as standard ROS2 messages.

For **surface navigation**, the simulator provides:

- *Inertial Measurement Unit (IMU)*: Provides high-frequency linear acceleration and angular velocity data, utilising the standard Gazebo sensor physics for realistic noise and bias modelling.
- *2D LiDAR*: Delivers planar range sensing for obstacle detection and SLAM applications via the Gazebo ray-tracing plugin to simulate light-based distance measurements.
- *Radar*: Simulates authentic maritime returns by convolving LiDAR point clouds with a range-dependent Point Spread Function (PSF) to create a rasterised display, following the methodology in [37].

For **underwater navigation**, the suite includes:

- *IMU & Magnetometer*: Provides high-frequency attitude estimation and heading reference data, utilising Gazebo's physics engine to simulate inertial and magnetic field vectors.
- *Depth Sensor*: Measures the vehicle's vertical position using the global z-coordinate as a high-fidelity proxy for physical pressure sensors, consistent with modeling frameworks such as [6], [8], [4].

Additional **mission-specific** sensors and modules include:

- *Perception*: Leverages Unity-based RGB cameras for visual recognition and high-fidelity marine radar to facilitate long-range situational awareness in complex maritime environments.
- *Operational Modules*: Integrates AIS (Automatic Identification System) data streams for vessel tracking and utilises a robust Waypoint Follower to enable autonomous path execution.

C. HITL Sensing

To enable seamless interaction between human operators and robotic agents, the simulator integrates sensors that capture human intent, motion, and cognitive state. This infrastructure allows users to control virtual entities or manage tactical interfaces through natural interaction modalities, as illustrated in Figure 3 and the accompanying video[2].

- *Eye Tracker*: A non-immersive gaze-tracking system monitors fixation points and pupil metrics to infer user attention and assess physiological factors such as fatigue and cognitive load.
- *Leap Motion*: A dedicated controller for high-fidelity hand and finger tracking, enabling gesture-based commands and precise human–machine interaction (HMI).
- *VR*: Provides an immersive first-person perspective, tracking 6-DOF head motion to facilitate real-time human input and environmental feedback.

Fig. 3: Left: user experimenting with VR in LOTUSim. Right: natural user-interaction interface.

TABLE VII: Metrics for model comparison

Metric	Definition / Formula	Interpretation / Key Value
Ekman vs. Gauss Ratio (MAE) (I_{MAE})	$I_{\mathrm{MAE}} = \dfrac{\mathrm{MAE}_{\mathrm{Ekman}}}{\mathrm{MAE}_{\mathrm{Gauss}}}$	Values < 1 indicate the Ekman model outperforms the Gauss–Markov model; lower is better
Ekman vs. Gauss Ratio (RMSE) (I_{RMSE})	$I_{\mathrm{RMSE}} = \dfrac{\mathrm{RMSE}_{\mathrm{Ekman}}}{\mathrm{RMSE}_{\mathrm{Gauss}}}$	Values < 1 indicate the Ekman model outperforms the Gauss–Markov model; lower is better
Mean Copernicus Current Magnitude (μ_{Cop})	$\mu_{\mathrm{Cop}} = \dfrac{1}{N}\sum_{i=1}^{N}\sqrt{u_{\mathrm{Cop,i}}^2 + v_{\mathrm{Cop,i}}^2}$	Provides a scale-independent normalisation factor for errors
Relative MAE ($\mathrm{Rel\_MAE}_M$)	$\mathrm{Rel\_MAE}_M = \dfrac{\mathrm{MAE}_M}{\mu_{\mathrm{Cop}}}$	Normalised MAE; lower values indicate better model performance
Relative RMSE ($\mathrm{Rel\_RMSE}_M$)	$\mathrm{Rel\_RMSE}_M = \dfrac{\mathrm{RMSE}_M}{\mu_{\mathrm{Cop}}}$	Normalised RMSE; lower values indicate better model performance

TABLE VIII: Comparison of error metrics between Ekman and Gauss–Markov models

	Depth Range	I_{MAE}	I_{RMSE}	$\mathrm{Rel\_MAE}_{\mathrm{Ekman}}$	$\mathrm{Rel\_MAE}_{\mathrm{Gauss}}$	$\mathrm{Rel\_RMSE}_{\mathrm{Ekman}}$	$\mathrm{Rel\_RMSE}_{\mathrm{Gauss}}$
4 Nov. 2023	Surface (0–100 m)	41.18%	31.47%	1.52	2.59	1.87	2.74
	Shallow (100–200 m)	73.23%	71.11%	1.85	6.92	2.05	7.11
	Mid-depth (200–500 m)	74.93%	74.11%	2.14	8.54	2.27	8.78
	Deep (500 m+)	80.29%	79.57%	2.03	10.30	2.16	10.57
3 June 2024	Surface (0–100 m)	44.73%	29.76%	1.20	2.18	1.63	2.32
	Shallow (100–200 m)	79.26%	78.48%	0.47	2.27	0.52	2.42
	Mid-depth (200–500 m)	81.03%	79.25%	0.49	2.61	0.57	2.76
	Deep (500 m+)	81.10%	79.72%	0.50	2.64	0.56	2.78
31 July 2024	Surface (0–100 m)	60.39%	59.71%	0.84	2.12	0.93	2.32
	Shallow (100–200 m)	79.28%	78.45%	0.43	2.09	0.48	2.24
	Mid-depth (200–500 m)	76.85%	76.27%	0.61	2.65	0.68	2.85
	Deep (500 m+)	40.57%	39.82%	0.79	1.32	0.90	1.49
7 August 2024	Surface (0–100 m)	55.37%	47.81%	1.07	2.40	1.30	2.49
	Shallow (100–200 m)	71.30%	70.92%	0.96	3.34	1.03	3.55
	Mid-depth (200–500 m)	72.56%	72.28%	0.96	3.50	1.03	3.73
	Deep (500 m+)	62.61%	61.07%	1.10	2.95	1.24	3.19
3 Oct. 2024	Surface (0–100 m)	46.40%	31.30%	0.93	1.74	1.27	1.86
	Shallow (100–200 m)	84.52%	83.66%	0.41	2.64	0.47	2.86
	Mid-depth (200–500 m)	80.03%	79.60%	0.58	2.88	0.65	3.16
	Deep (500 m+)	82.88%	82.18%	0.44	2.55	0.50	2.82

VI. Simulation-to-Real

In [38], simulation results obtained with LOTUSim were successfully transferred to a physical BlueROV2 platform, demonstrating the simulator's sim-to-real capability. In that study, a fault-tolerant control strategy for the BlueROV2 was first designed and evaluated in simulation. Subsequent real-world experiments conducted in a test pool (Figure 4)) validated the simulated behavior: the controller dynamically adapted motor commands in response to faults, without relying on explicit fault diagnosis.

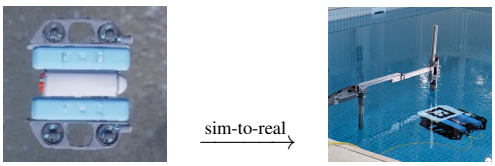

Fig. 4: Sim-to-real transfer for the BlueROV2 Heavy: (left) in LOTUSim; (right) in the test pool.

In the present work, we exploit LOTUSim's ability to accelerate the real-time factor (RTF) to enable efficient AI training. As illustrated in Figure 5, LOTUSim achieves an RTF exceeding 20 for a single drone. Even when scaling up to a swarm of 25 drones, the RTF remains above 1, allowing simulations to run faster than real-time.

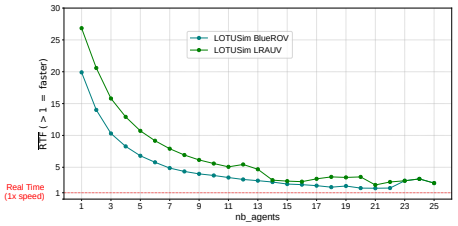

Fig. 5: RTF vs. number of agents (30ms)

VII. Conclusion

This paper introduced LOTUSim, an open-source simulator bridging the gap between maritime robotics and human-centric naval operations. By integrating aerial, surface, and underwater assets into a distributed, multi-user framework, it enables high-fidelity mission rehearsal and real-time HITL applications. Unlike existing tools, LOTUSim features a validated Ekman-inspired current model, providing superior environmental realism while maintaining scalability for large fleets and immersive HITL interfaces. The results demonstrate that LOTUSim supports complex scenarios

without sacrificing real-time responsiveness. Future work will focus on multi-agent coordination, scalable deployment, and improved sensor and environmental fidelity, positioning LOTUSim as a flexible platform for maritime research.

REFERENCES

[1] H. Chen, Y. Xu, Y. Ren, Y. Ye, X. Li, N. Ding, P. Cong, Z. Wang, B. Liu, Y. Chen, Z. Dou, X. Leng, M. Li, Y. Ma, and C. Tu, "Symbiosim: Human-in-the-loop simulation platform for bidirectional continuing learning in human–robot interaction," *arXiv preprint arXiv:2502.07358*, 2025. [Online]. Available: https://arxiv.org/abs/2502.07358

[2] J. Song, H. Ma, O. Bagoren, A. V. Sethuraman, Y. Zhang, and K. A. Skinner, "Oceansim: A gpu-accelerated underwater robot perception simulation framework," *arXiv preprint arXiv:2503.01074*, 2025. [Online]. Available: https://arxiv.org/abs/2503.01074

[3] S. Chu, Z. Huang, Y. Li, M. Lin, D. Li, I. Carlucho, Y. R. Petillot, and C. Yang, "Marinegym: A high-performance reinforcement learning platform for underwater robotics," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 17 146–17 153.

[4] M. Grimaldi, P. Cieslak, E. Ochoa, V. Bharti, H. Rajani, I. Carlucho, M. Koskinopoulou, Y. R. Petillot, and N. Gracias, "Stonefish: Supporting machine learning research in marine robotics," 2025. [Online]. Available: https://arxiv.org/abs/2502.11887

[5] E. Potokar, S. Ashford, M. Kaess, and J. Mangelson, "HoloOcean: An underwater robotics simulator," in *Proc. IEEE Intl. Conf. on Robotics and Automation, ICRA*, Philadelphia, PA, USA, May 2022.

[6] B. Romrell, A. Austin, B. Meyers, R. Anderson, C. Noh, and J. G. Mangelson, "A preview of holoocean 2.0," 2025. [Online]. Available: https://arxiv.org/abs/2510.06160

[7] M. Paravisi, D. H. Santos, V. Jorge, G. Heck, L. M. Gonçalves, and A. Amory, "Unmanned surface vehicle simulator with realistic environmental disturbances," *Sensors*, vol. 19, no. 5, p. 1068, Mar. 2019. [Online]. Available: http://dx.doi.org/10.3390/s19051068

[8] T. R. Player, A. Chakravarty, M. M. Zhang, B. Y. Raanan, B. Kieft, Y. Zhang, and B. Hobson, "From concept to field tests: Accelerated development of multi-auv missions using a high-fidelity faster-than-real-time simulator," *arXiv preprint arXiv:2311.10377*, 2023. [Online]. Available: https://arxiv.org/abs/2311.10377

[9] M. M. M. Manhaes, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, "Uuv simulator: A gazebo-based package for underwater intervention and multi-robot simulation," in *OCEANS 2016 MTS/IEEE Monterey*. IEEE, Sep. 2016, p. 1–8. [Online]. Available: http://dx.doi.org/10.1109/OCEANS.2016.7761080

[10] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, and B. Bingham, "Dave aquatic virtual environment: Toward a general underwater robotics simulator," 2022. [Online]. Available: https://arxiv.org/abs/2209.02862

[11] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "Airsim: High-fidelity visual and physical simulation for autonomous vehicles," 2017. [Online]. Available: https://arxiv.org/abs/1705.05065

[12] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, ser. IROS-04, vol. 3. IEEE, 2004, p. 2149–2154. [Online]. Available: http://dx.doi.org/10.1109/IROS.2004.1389727

[13] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Computers in Physics*, vol. 12, no. 6, p. 620–631, Nov. 1998. [Online]. Available: http://dx.doi.org/10.1063/1.168744

[14] N. Kakavitsas, A. Willis, R. Jacobik, M. Uddin, and A. Wolek, "Quadrotor flight simulation in a cfd-generated urban wind field," Mar. 2024. [Online]. Available: http://dx.doi.org/10.36227/techrxiv.171085159.95135995/v1

[15] J. Song, H. Ma, O. Bagoren, A. V. Sethuraman, Y. Zhang, and K. A. Skinner, "Oceansim: A gpu-accelerated underwater robot perception simulation framework," 2025. [Online]. Available: https://arxiv.org/abs/2503.01074

[16] A. Amer, O. Álvarez Tuñón, H. I. Ugurlu, J. le Fevre Sejersen, Y. Brodskiy, and E. Kayacan, "Unav-sim: A visually realistic underwater robotics simulator and synthetic data-generation framework," 2023. [Online]. Available: https://arxiv.org/abs/2310.11927

[17] P. Katara, M. Khanna, H. Nagar, and A. Panaiyappan, "Open source simulator for unmanned underwater vehicles using ros and unity3d," in *2019 IEEE Underwater Technology (UT)*. IEEE, Apr. 2019, p. 1–7. [Online]. Available: http://dx.doi.org/10.1109/UT.2019.8734309

[18] J. Ferber and G. Weiss, *Multi-agent systems: an introduction to distributed artificial intelligence*. Addison-wesley Reading, 1999, vol. 1.

[19] J. Ko *et al.*, "Ros2 and ignition: Towards scalable simulation of distributed robotic systems," in *IEEE/SICE International Symposium on System Integration (SII)*, 2020, pp. 378–383. [Online]. Available: https://ieeexplore.ieee.org/document/9075123

[20] S. Weech, S. Kenny, and M. Barnett-Cowan, "Effect of frame rate on user experience, performance and simulator sickness in virtual reality," *Human Factors*, vol. 65, no. 4, p. 678–692, 2023.

[21] A. e. a. Moss, "Perceptual thresholds for display lag in a real visual environment," *Human Factors*, 2010.

[22] S. Dumnich, W. Birmingham, and B. Wolfe, "Embracing the lag: Real-time challenges in multi-agent systems," in *Proceedings of the AAAI Fall Symposium Series (FSS-23)*, 2023.

[23] K. Hunkins, "Ekman drift currents in the arctic ocean," *Deep Sea Research and Oceanographic Abstracts*, vol. 13, no. 4, p. 607–620, Aug. 1966. [Online]. Available: http://dx.doi.org/10.1016/0011-7471(66)90592-4

[24] S. POND and G. L. PICKARD, *Currents with Friction; Wind-driven Circulation*. Elsevier, 1983, p. 100–162. [Online]. Available: http://dx.doi.org/10.1016/B978-0-08-057054-9.50015-8

[25] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. Wiley, Apr. 2011. [Online]. Available: http://dx.doi.org/10.1002/9781119994138

[26] A. Babarit and M. Charlou, "Cn-aeromodels: A c++ implementation of aerodynamic models for wind propulsion systems of cargo ships," *Journal of Open Source Software*, vol. 9, no. 102, p. 6940, Oct. 2024. [Online]. Available: http://dx.doi.org/10.21105/joss.06940

[27] Y. Goda, *Random Seas and Design of Maritime Structures*. WORLD SCIENTIFIC, Jun. 2010. [Online]. Available: http://dx.doi.org/10.1142/7425

[28] G. Rodenbusch and G. Forristall, "An empirical model for random directional wave kinematics near the free surface," in *Offshore Technology Conference*, ser. 86OTC. OTC, May 1986. [Online]. Available: http://dx.doi.org/10.4043/5097-MS

[29] R. Stewart, *Introduction to Physical Oceanography*. University Press of Florida, 2009. [Online]. Available: https://open.umn.edu/opentextbooks/textbooks/20

[30] S. Hautala, "Physics across oceanography: Fluid mechanics and waves," *University of Washington*, 2020.

[31] A. Constantin and R. S. Johnson, "Ekman-type solutions for shallow-water flows on a rotating sphere: A new perspective on a classical problem," *Physics of Fluids*, vol. 31, no. 2, Feb. 2019. [Online]. Available: http://dx.doi.org/10.1063/1.5083088

[32] B. Cushman-Roisin and J.-M. Beckers, *Introduction to geophysical fluid dynamics: physical and numerical aspects*, ser. International geophysics. Elsevier, 2011. [Online]. Available: https://www.elsevier.com/books/introduction-to-geophysical-fluid-dynamics/cushman-roisin/978-0-12-088759-0

[33] M. Curcic and B. K. Haus, "Revised estimates of ocean surface drag in strong winds," *Geophysical Research Letters*, vol. 47, no. 10, May 2020. [Online]. Available: http://dx.doi.org/10.1029/2020GL087647

[34] C. Bayindir, "Predicting the ocean currents using deep learning," 2019. [Online]. Available: https://arxiv.org/abs/1906.08066

[35] A. Sinha and R. Abernathey, "Estimating ocean surface currents with machine learning," *Frontiers in Marine Science*, vol. 8, Jun. 2021. [Online]. Available: http://dx.doi.org/10.3389/fmars.2021.672477

[36] D. Menaka and S. Gauni, "Prediction of dominant ocean parameters for sustainable marine environment," *IEEE Access*, vol. 9, p. 146578–146591, 2021. [Online]. Available: http://dx.doi.org/10.1109/ACCESS.2021.3122237

[37] B. Lesy, S. Herremans, R. Kerstens, J. Steckel, W. Daems, S. Mercelis, and A. Anwar, "Asvsim (airsim for surface vehicles): A high-fidelity simulation framework for autonomous surface vehicle research," 2025. [Online]. Available: https://arxiv.org/abs/2506.22174

[38] K. Lagattu, E. Artusi, P. E. Santos, K. Sammut, G. Le Chenadec, and B. Clement, "Control reallocation using deep reinforcement learning for actuator fault recovery of an autonomous underwater vehicle," in *2025 IEEE ICRA*, 2025, pp. 2291–2297.